信 息 工 程 学 院

《深度学习》实验报告

学院：信息工程学院　**班级**：数科 2102　**姓名**：陈　**学号**:202101　**成绩**:

一、实验题目、

卷积神经网络

二、实验目的

认识 Tensorflow；认识 TensorFlow 变量；进行基本的 TensorFlow 的应用；学习深度神经网络的搭建与应用；学习卷积神经网络的搭建与训练

三、实验平台

希冀平台

四、实验过程及步骤

1. 认识 Tensorflow

【练习】构建图

构建图的第一步, 是创建源 op (source op). 源 op 不需要任何输入, 例如 常量 (Constant). 源 op 的输出被传递给其它 op 做运算.

Python 库中, op 构造器的返回值代表被构造出的 op 的输出, 这些返回值可以传递给其它 op 构造器作为输入.

TensorFlow Python 库有一个默认图 (default graph), op 构造器可以为其增加节点. 这个默认图对 许多程序来说已经足够用了.

In [1]:
```
import tensorflow as tf
# 创建一个常量 op, 产生一个 1x2 矩阵. 这个 op 被作为一个节点
# 加到默认图中.
#
matrix1 = tf.constant([[1.,2.]])
# 构造器的返回值代表该常量 op 的返回值.

# 创建另外一个常量 op, 产生一个 2x1 矩阵.
matrix2 = tf.constant([[3.],[4.]])
# 创建一个矩阵乘法 matmul op , 把 'matrix1' 和 'matrix2' 作为输入.
ans = tf.matmul(matrix1, matrix2)
# 返回值 'product' 代表矩阵乘法的结果.
print(ans)
```

```
/opt/conda/lib/python3.6/site-packages/h5py/__init__.py:36: FutureWarning: Conversion of the second argument of issubdtype from `float`
to `np.floating` is deprecated. In future, it will be treated as `np.float64 == np.dtype(float).type`.
  from .._conv import register_converters as _register_converters
```

```
Tensor("MatMul:0", shape=(1, 1), dtype=float32)
```

默认图现在有三个节点, 两个 constant() op, 和一个matmul() op. 为了真正进行矩阵相乘运算, 并得到矩阵乘法的 结果, 你必须在会话里启动这个图.

实验报告的内容与格式按任课教师的要求书写。

【练习】 在一个会话中启动图

构造阶段完成后, 才能启动图. 启动图的第一步是创建一个 Session 对象, 如果无任何创建参数, 会话构造器将启动默认图.

In [2]:
```
# 启动默认图.
sess = tf.Session()

# 调用 sess 的 'run()' 方法来执行矩阵乘法 op, 传入 'product' 作为该方法的参数.
# 上面提到, 'product' 代表了矩阵乘法 op 的输出, 传入它是向方法表明, 我们希望取回
# 矩阵乘法 op 的输出.
res = sess.run(ans)
print(res)
# 整个执行过程是自动化的, 会话负责传递 op 所需的全部输入. op 通常是并发执行的.
#
# 函数调用 'run(product)' 触发了图中三个 op (两个常量 op 和一个矩阵乘法 op) 的执行.
#
# 返回值 'result' 是一个 numpy `ndarray` 对象.

# ==> [[ 12.]]

# 任务完成, 关闭会话.
sess.close()
```

```
[[11.]]
```

【练习】交互式使用 ¶

文档中的 Python 示例使用一个会话 Session 来 启动图, 并调用 Session.run() 方法执行操作.

为了便于使用诸如 IPython 之类的 Python 交互环境, 可以使用 InteractiveSession 代替 Session 类, 使用 Tensor.eval() 和 Operation.run() 方法代替 Session.run(). 这样可以避免使用一个变量来持有会话.

In [3]:
```
# 进入一个交互式 TensorFlow 会话.
import tensorflow as tf
sess = tf.InteractiveSession()

x = tf.Variable([1.0, 2.0])
a = tf.constant([3.0, 3.0])

# 使用初始化器 initializer op 的 run() 方法初始化 'x'
x.initializer.run()

# 增加一个减法 sub op, 从 'x' 减去 'a'. 运行减法 op, 输出结果
sub = tf.subtract(x,a)
print(sub.eval())
# ==> [-2. -1.]
sess.close()
```

```
[-2. -1.]
```

【练习】变量

Variables for more details. 变量维护图执行过程中的状态信息. 如何使用变量实现一个简单的计数器.

In [4]:
```
# 创建一个变量, 初始化为标量 0.
state = tf.Variable(0)

# 创建一个 op, 其作用是使 state 增加 1
a = tf.constant(1)
res = tf.add(state,a)
new_state = tf.assign(state,res)


# 启动图后, 变量必须先经过`初始化` (init) op 初始化,
# 首先必须增加一个`初始化` op 到图中.
init_op = tf.initialize_all_variables()

with tf.Session() as sess:
    sess.run(init_op)
    print(sess.run(state))
    for i in range(3):
        sess.run(new_state)
        print(sess.run(state))

sess.close()

# 启动图, 运行 op

  # 运行 'init' op

  # 打印 'state' 的初始值

  # 运行 op, 更新 'state', 并打印 'state'


# 输出:

# 0
# 1
# 2
# 3
```

```
WARNING: Logging before flag parsing goes to stderr.
W1115 00:38:13.972987 140708925531968 deprecation.py:323] From /opt/conda/lib/python3.6/site-packages/tensorflow_core/python/util/tf_should_
use.py:198: initialize_all_variables (from tensorflow.python.ops.variables) is deprecated and will be removed after 2017-03-02.
Instructions for updating:
Use `tf.global_variables_initializer` instead.
```

```
0
1
2
3
```

【练习】Fetch

为了取回操作的输出内容, 可以在使用 Session 对象的 run() 调用 执行图时, 传入一些 tensor, 这些 tensor 会帮助你取回结果. 在之前的例子里, 我们只取回了单个节点 state, 但是你也可以取回多个 tensor:

In [5]:
```
input1 = tf.constant([3.])
input2 = tf.constant([2.])
input3 = tf.constant([5.])
intermed = tf.add(input2, input3)
mul = tf.multiply(input1, intermed)

with tf.Session() as sess:
  result = sess.run([mul, intermed])
  print(result)


# 输出:
# [array([ 21.], dtype=float32), array([ 7.], dtype=float32)]
```

```
[array([21.], dtype=float32), array([7.], dtype=float32)]
```

需要获取的多个 tensor 值，在 op 的一次运行中一起获得（而不是逐个去获取 tensor）。

【练习】Feed

上述示例在计算图中引入了 tensor, 以常量或变量的形式存储. TensorFlow 还提供了 feed 机制, 该机制 可以临时替代图中的任意操作中的 tensor 可以对图中任何操作提交补丁, 直接插入一个 tensor.

feed 使用一个 tensor 值临时替换一个操作的输出结果. 你可以提供 feed 数据作为 run() 调用的参数. feed 只在调用它的方法内有效, 方法结束, feed 就会消失. 最常见的用例是将某些特殊的操作指定为 "feed" 操作, 标记的方法是使用 tf.placeholder() 为这些操作创建占位符.

In [6]:
```
input1 = tf.placeholder(tf.float32)
input2 = tf.placeholder(tf.float32)
output = tf.multiply(input1, input2)

with tf.Session() as sess:
  print(sess.run([output], feed_dict={input1:[7.], input2:[2.]}))
# 输出:
# [array([ 14.], dtype=float32)]
```

```
[array([14.], dtype=float32)]
```

for a larger-scale example of feeds. 如果没有正确提供 feed, placeholder() 操作将会产生错误.

2. TensorFlow 变量介绍

【练习】创建

当创建一个变量时，你将一个张量作为初始值传入构造函数Variable()。TensorFlow提供了一系列操作符来初始化张量，初始值是常量或是随机值。

注意，所有这些操作符都需要你指定张量的shape。那个形状自动成为变量的shape。变量的shape通常是固定的，但TensorFlow提供了高级的机制来重新调整其行列数。

In [1]:
```
import tensorflow as tf
# Create two variables.
weights = tf.Variable(tf.random_normal([784, 200], stddev=0.35),
                      name="weights")
### Start Code Here ###
#创建一个biases变量，初始化为零矩阵向量，大小为[200]
biases = tf.Variable(tf.zeros([200]), name="biases")

### End Code Here ###
```

In [2]:
```
### Start Code Here ###
# Create a variable with a random value.shape:[784,200],name:weights
weights = tf.Variable(tf.random_normal([784, 200]), name="weights")
# Create another variable with the same value as 'weights'.named w2
w2 = tf.Variable(weights.initial_value, name="w2")
# Create another variable with twice the value of 'weights' named w_twice
w_twice = tf.Variable(weights.initial_value*2, name="w_twice")
### End Code Here ###
```

In [3]:
```
weights
```

Out[3]:
```
<tf.Variable 'weights_1:0' shape=(784, 200) dtype=float32_ref>
```

【练习】保存变量

用tf.train.Saver()创建一个Saver来管理模型中的所有变量。

In [4]:

```
# Create some variables.
v1 = tf.Variable(tf.ones([200]), name="v1")
v2 = tf.Variable(tf.zeros([200]), name="v2")
#...
### Start Code Here ###
# Add an op to initialize the variables.
init_op = tf.global_variables_initializer()
# Add ops to save a# Add ops to save and restore all the variables.
saver = tf.train.Saver()
### End Code Here ###

# Later, launch the model, initialize the variables, do some work, save the
# variables to disk.
with tf.Session() as sess:
    sess.run(init_op)
    # Do some work with the model.

    ### Start Code Here ###
    # Save the variables to disk.Save to save_path.
    save_path = saver.save(sess, "./tmp/model.ckpt")

    ### End Code Here ###
    print ("Model saved in file: ", save_path)
```

```
Model saved in file:  ./tmp/model.ckpt
```

【练习】恢复变量

用同一个Saver对象来恢复变量。注意，当你从文件中恢复变量时，不需要事先对它们做初始化。

In [5]:

```
# Create some variables.
tf.reset_default_graph()
v1 = tf.Variable(tf.ones([200]),name="v1")
v2 = tf.Variable(tf.ones([200]),name="v2")
#...
# Add ops to save and restore all the variables.
saver = tf.train.Saver()

# Later, launch the model, use the saver to restore variables from disk, and
# do some work with the model.
with tf.Session() as sess:
    ### Start Code Here ###
    # Restore variables from disk."./tmp/model.ckpt"
    saver.restore(sess, "./tmp/model.ckpt")

    ### End Code Here ###
    print ("Model restored.")
  # Do some work with the model
  #...
```

```
Model restored.
```

In [6]:

```
v1
```

Out[6]:

```
<tf.Variable 'v1:0' shape=(200,) dtype=float32_ref>
```

【练习】选择存储和恢复哪些变量

如果你不给tf.train.Saver()传入任何参数，那么saver将处理graph中的所有变量。其中每一个变量都以变量创建时传入的名称被保存。

有时候在检查点文件中明确定义变量的名称很有用。举个例子，你也许已经训练得到了一个模型，其中有个变量命名为"weights"，你想把它的值恢复到一个新的变量"params"中。

有时候仅保存和恢复模型的一部分变量很有用。再举个例子，你也许训练得到了一个5层神经网络，现在想训练一个6层的新模型，可以将之前5层模型的参数导入到新模型的前5层中。

你可以通过给tf.train.Saver()构造函数传入Python字典，很容易地定义需要保持的变量及对应名称：键对应使用的名称，值对应被管理的变量。

注意：

如果需要保存和恢复模型变量的不同子集，可以创建任意多个saver对象。同一个变量可被列入多个saver对象中，只有当saver的restore()函数被运行时，它的值才会发生改变。 如果你仅在session开始时恢复模型变量的一个子集，你需要对剩下的变量执行初始化op。

In [7]:

```
# Create some variables.
v1 = tf.Variable(tf.ones([200]), name="v1")
v2 = tf.Variable(tf.ones([200]), name="v2")
...

### Start Code Here ###
## Add ops to save and restore only 'v2' using the name "my_v2"
saver = tf.train.Saver({"my_v2":v2})

### End Code Here ###

# Use the saver object normally after that.
...
```

Out[7]:

```
Ellipsis
```

3. TensorFlow 的应用

导入TensorFlow库

In [1]:
```
import numpy as np
import h5py
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.python.framework import ops
import tf_utils
import time

%matplotlib inline
np.random.seed(1)
```

【练习】定义和初始化会话

In [2]:
```
### Start Code Here ###
#定义y_hat为固定值36
y_hat = tf.constant(36,name="y_hat")
#定义y为固定值39
y = tf.constant(39,name="y")
#为损失函数创建一个变量
loss = tf.Variable((y-y_hat)**2,name="loss" )
#运行之后的初始化(ession.run(init))
init = tf.global_variables_initializer()
#损失变量将被初始化并准备计算

#创建一个session并打印输出
with tf.Session() as session:
    #初始化变量
    session.run(init)
    #打印损失值
    print(session.run(loss))
### End Code Here ###
```

```
9
```

运行Session，此时，之前编写操作都会在这一步运行。

因此，当我们为损失函数创建一个变量时，我们简单地将损失定义为其他数量的函数，但没有评估它的价值。为了评估它，我们需要运行init=tf.global_variables_initializer()，初始化损失变量，在最后一行，我们最后能够评估损失的值并打印它的值。

In [3]:
```
a = tf.constant(2)
b = tf.constant(10)
c = tf.multiply(a,b)

print(c)
```

```
Tensor("Mul:0", shape=(), dtype=int32)
```

正如预料中一样，我们并没有看到结果20，不过我们得到了一个Tensor类型的变量，没有维度，数字类型为int32。

我们之前所做的一切都只是把这些东西放到了一个"计算图(computation graph)"中，而我们还没有开始运行这个计算图，为了实际计算这两个数字，我们需要创建一个会话并运行它：

In [4]:
```
sess = tf.Session()

print(sess.run(c))
```

```
20
```

【练习】创建占位符

接下来，我们需要了解一下占位符（placeholders）。占位符是一个对象，它的值只能在稍后指定，要指定占位符的值，可以使用一个feed字典（feed_dict变量）来传入。

接下来，我们为x创建一个占位符，这将允许我们在稍后运行会话时传入一个数字。

In [5]:
```
### Start Code Here ###
##利用feed_dict来改变x的值
#为x创建一个占位符，类型为tf.int64
x = tf.compat.v1.placeholder(tf.int64, name='x')

### End Code Here ###

print(sess.run(2 * x,feed_dict={x:3}))
sess.close()
```

```
6
```

【练习】线性函数 ¶

让我们通过计算以下等式来开始编程：$Y = WX + b$,W和X是随机矩阵，b是随机向量。

我们计算$WX + b$，其中W，X和b是从随机正态分布中抽取的。 W的维度是（4,3），X是（3,1），b是（4,1）。

我们开始定义一个shape=（3,1）的常量X：

In [6]:
```
def linear_function():
    """
    实现一个线性功能：
        初始化W，类型为tensor的随机变量，维度为(4,3)
        初始化X，类型为tensor的随机变量，维度为(3,1)
        初始化b，类型为tensor的随机变量，维度为(4,1)
    返回：
        result - 运行了session后的结果，运行的是Y = WX + b

    """
### Start Code Here ###
#实现这个函数
    np.random.seed(114514)
    X = np.random.randn(3,1)
    W = np.random.randn(4,3)
    b = np.random.randn(4,1)

    Y = tf.add(tf.matmul(W,X),b)
    sess = tf.compat.v1.Session()
    result = sess.run(Y)
    sess.close()
    return result

### End Code Here ###
```

In [7]:
```
print("result = " +  str(linear_function()))
```

```
result = [[-0.18369607]
 [-0.61044329]
 [ 0.10923952]
 [-0.19370181]]
```

```
    result = sess.run(...,feed_dict = {...})
    sess.close()
```

方法2

```
    with tf.Session as sess:
        result = sess.run(...,feed_dict = {...})
```

In [8]:
```
def sigmoid(z):
    """
    实现使用sigmoid函数计算z

    参数：
        z - 输入的值，标量或矢量

    返回：
        result - 用sigmoid计算z的值

    """

    #创建一个占位符x，名字叫"x"
    x = tf.placeholder(tf.float32,name="x")

    ### Start Code Here ###
    #计算sigmoid(z)
    sigmoid = tf.sigmoid(x)
    #创建一个会话，使用方法二
    with tf.compat.v1.Session() as sess:
        result = sess.run(sigmoid,feed_dict={x:z})
    ### End Code Here ###

    return result
```

In [9]:
```
print ("sigmoid(0) = " + str(sigmoid(0)))
print ("sigmoid(12) = " + str(sigmoid(12)))
```

```
sigmoid(0) = 0.5
sigmoid(12) = 0.9999938
```

In [10]:
```
def one_hot_matrix(lables,C):
    """
    创建一个矩阵，其中第i行对应第i个类号，第j列对应第j个训练样本
    所以如果第j个样本对应着第i个标签，那么entry (i,j)将会是1

    参数:
        lables - 标签向量
        C - 分类数

    返回:
        one_hot - 独热矩阵

    """

    #创建一个tf.constant，赋值为C，名字叫C
    C = tf.constant(C,name="C")


    ### Start Code Here ###
    ##使用tf.one_hot，注意一下axis
    one_hot_matrix = tf.one_hot(indices=lables, depth=C, axis=0)

    ### End Code Here ###

    #创建一个session
    sess = tf.Session()

    #运行session
    one_hot = sess.run(one_hot_matrix)

    #关闭session
    sess.close()

    return one_hot
```

In [11]:
```
labels = np.array([1,2,3,0,2,1])
one_hot = one_hot_matrix(labels,C=4)
print(str(one_hot))
```

```
[[0. 0. 0. 1. 0. 0.]
 [1. 0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 1. 0.]
 [0. 0. 1. 0. 0. 0.]]
```

【练习】初始化为0或1

现在我们将学习如何用0或者1初始化一个向量，我们要用到tf.ones()和tf.zeros()，给定这些函数一个维度值那么它们将会返回全是1或0的满足条件的向量/矩阵。

In [12]:
```
def ones(shape):
    """
    创建一个维度为shape的变量，其值全为1

    参数:
        shape - 你要创建的数组的维度

    返回:
        ones - 只包含1的数组
    """
    ### Start Code Here ###
    ##使用tf.ones()
    ones = tf.ones(shape)

    ### End Code Here ###

    #创建会话
    sess = tf.Session()

    #运行会话
    ones = sess.run(ones)

    #关闭会话
    sess.close()

    return ones
```

In [13]:
```
print ("ones = " + str(ones([3])))
```

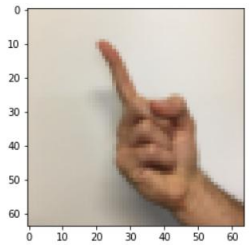
```
ones = [1. 1. 1.]
```

加载数据集

In [14]:
```
X_train_orig , Y_train_orig , X_test_orig , Y_test_orig , classes = tf_utils.load_dataset()
```

In [15]:
```
index = 11
plt.imshow(X_train_orig[index])
print("Y = " + str(np.squeeze(Y_train_orig[:,index])))
```

```
Y = 1
```



In [16]:
```
### Start Code Here ###
#X_train_orig、X_test_orig数据扁平化成X_train_flatten、X_test_flatten
X_train_flatten = X_train_orig.reshape(X_train_orig.shape[0],-1).T
X_test_flatten = X_test_orig.reshape(X_test_orig.shape[0],-1).T
#归一化数据成X_train、X_test
X_train = X_train_flatten / 255
X_test = X_test_flatten / 255
#Y_train_orig、Y_test_orig转换为独热矩阵Y_train、Y_test
Y_train = tf_utils.convert_to_one_hot(Y_train_orig,6)
Y_test = tf_utils.convert_to_one_hot(Y_test_orig,6)

### End Code Here ###


print("训练集样本数 = " + str(X_train.shape[1]))
print("测试集样本数 = " + str(X_test.shape[1]))
print("X_train.shape: " + str(X_train.shape))
print("Y_train.shape: " + str(Y_train.shape))
print("X_test.shape: " + str(X_test.shape))
print("Y_test.shape: " + str(Y_test.shape))
```

```
训练集样本数 = 1080
测试集样本数 = 120
X_train.shape: (12288, 1080)
Y_train.shape: (6, 1080)
X_test.shape: (12288, 120)
Y_test.shape: (6, 120)
```

创建placeholders

我们的第一项任务是为X和Y创建占位符，这将允许我们稍后在运行会话时传递您的训练数据。

In [17]:
```
def create_placeholders(n_x,n_y):
    """
    为TensorFlow会话创建占位符
    参数：
        n_x - 一个实数，图片向量的大小（64*64*3 = 12288）
        n_y - 一个实数，分类数（从0到5，所以n_y = 6）

    返回：
        X - 一个数据输入的占位符，维度为[n_x, None]，dtype = "float"
        Y - 一个对应输入的标签的占位符，维度为[n_Y,None]，dtype = "float"

    提示：
        使用None，因为它让我们可以灵活处理占位符提供的样本数量。事实上，测试/训练期间的样本数量是不同的。

    """

    ### Start Code Here ###
    #为X，Y创建占位符
    X = tf.compat.v1.placeholder(tf.float32, [n_x, None], name="X")
    Y = tf.compat.v1.placeholder(tf.float32, [n_y, None], name="Y")

    ### End Code Here ###


    return X, Y
```

In [18]:
```
X, Y = create_placeholders(12288, 6)
print("X = " + str(X))
print("Y = " + str(Y))
```

```
X = Tensor("X_3:0", shape=(12288, ?), dtype=float32)
Y = Tensor("Y_1:0", shape=(6, ?), dtype=float32)
```

初始化参数

初始化tensorflow中的参数，我们将使用Xavier初始化权重和用零来初始化偏差。

tf.Variable() 每次都在创建新对象，对于get_variable()来说，对于已经创建的变量对象，就把那个对象返回，如果没有创建变量对象的话，就创建一个新的。

In [19]:
```
def initialize_parameters():
    """
    初始化神经网络的参数，参数的维度如下：
        W1 : [25, 12288]
        b1 : [25, 1]
        W2 : [12, 25]
        b2 : [12, 1]
        W3 : [6, 12]
        b3 : [6, 1]

    返回：
        parameters - 包含了W和b的字典


    """

    tf.set_random_seed(1) #指定随机种子

    W1 = tf.get_variable("W1",[25,12288],initializer=tf.contrib.layers.xavier_initializer(seed=1))
    b1 = tf.get_variable("b1",[25,1],initializer=tf.zeros_initializer())
    W2 = tf.get_variable("W2", [12, 25], initializer = tf.contrib.layers.xavier_initializer(seed=1))
    b2 = tf.get_variable("b2", [12, 1], initializer = tf.zeros_initializer())
    ### Start Code Here ###
    #依照以上方法初始化参数W3,b3。w3:[6,12],b3:[6,1]
    W3 = tf.get_variable("W3",[6,12],initializer=tf.contrib.layers.xavier_initializer(seed=1))
    b3 = tf.get_variable("b3",[6,1],initializer=tf.zeros_initializer())

    ### End Code Here ###


    parameters = {"W1": W1,
                  "b1": b1,
                  "W2": W2,
                  "b2": b2,
                  "W3": W3,
                  "b3": b3}

    return parameters
```

In [20]:
```
tf.reset_default_graph() #用于清除默认图形堆栈并重置全局默认图形。

with tf.Session() as sess:
    parameters = initialize_parameters()
    print("W1 = " + str(parameters["W1"]))
    print("b1 = " + str(parameters["b1"]))
    print("W2 = " + str(parameters["W2"]))
    print("b2 = " + str(parameters["b2"]))
```

```
WARNING: Logging before flag parsing goes to stderr.
W1115 01:24:14.882897 139999335434048 lazy_loader.py:50]
The TensorFlow contrib module will not be included in TensorFlow 2.0.
For more information, please see:
  * https://github.com/tensorflow/community/blob/master/rfcs/20180907-contrib-sunset.md
  * https://github.com/tensorflow/addons
  * https://github.com/tensorflow/io (for I/O related ops)
If you depend on functionality not listed there, please file an issue.
```

```
W1 = <tf.Variable 'W1:0' shape=(25, 12288) dtype=float32_ref>
b1 = <tf.Variable 'b1:0' shape=(25, 1) dtype=float32_ref>
W2 = <tf.Variable 'W2:0' shape=(12, 25) dtype=float32_ref>
b2 = <tf.Variable 'b2:0' shape=(12, 1) dtype=float32_ref>
```

线性输出层的输出作为计算损失函数的输入，所以不需要A3.

In [21]:
```
def forward_propagation(X,parameters):
    """
    实现一个模型的前向传播，模型结构为LINEAR -> RELU -> LINEAR -> RELU -> LINEAR -> SOFTMAX

    参数：
        X - 输入数据的占位符，维度为（输入节点数量，样本数量）
        parameters - 包含了W和b的参数的字典

    返回：
        Z3 - 最后一个LINEAR节点的输出

    """

    W1 = parameters['W1']
    b1 = parameters['b1']
    W2 = parameters['W2']
    b2 = parameters['b2']
    W3 = parameters['W3']
    b3 = parameters['b3']


    ### Start Code Here ###
    #使用tensorflow的op函数进行实现
    # Z1 = np.dot(W1, X) + b1
    Z1 = tf.add(tf.matmul(W1,X),b1)
    # A1 = relu(Z1)
    A1 = tf.nn.relu(Z1)
    # Z2 = np.dot(W2, a1) + b2
    Z2 = tf.add(tf.matmul(W2,A1),b2)
    # A2 = relu(Z2)
    A2 = tf.nn.relu(Z2)
    # Z3 = np.dot(W3,Z2) + b3
    Z3 = tf.add(tf.matmul(W3,A2),b3)
    ### End Code Here ###

    return Z3
```

In [22]:
```
tf.reset_default_graph() #用于清除默认图形堆栈并重置全局默认图形。
with tf.Session() as sess:
    X,Y = create_placeholders(12288,6)
    parameters = initialize_parameters()
    Z3 = forward_propagation(X,parameters)
    print("Z3 = " + str(Z3))
```

```
Z3 = Tensor("Add_2:0", shape=(6, ?), dtype=float32)
```

可能已经注意到前向传播不会输出任何cache，当我们完成反向传播的时候你就会明白了。

In [23]:
```
def compute_cost(Z3,Y):
    """
    计算成本

    参数：
        Z3 - 前向传播的结果
        Y - 标签，一个占位符，和Z3的维度相同

    返回：
        cost - 成本值


    """
    logits = tf.transpose(Z3) #转置
    labels = tf.transpose(Y)  #转置

    ### Start Code Here ###
    #实现cost函数的计算
    cost = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=logits,labels=labels))

    ### End Code Here ###

    return cost
```

In [24]:
```
tf.reset_default_graph()

with tf.Session() as sess:
    X,Y = create_placeholders(12288,6)
    parameters = initialize_parameters()
    Z3 = forward_propagation(X,parameters)
    cost = compute_cost(Z3,Y)
    print("cost = " + str(cost))
```

```
W1115 01:24:15.710309 139999335434048 deprecation.py:323] From <ipython-input-23-ff7c6ca251ca>:19: softmax_cross_entropy_with_logit
s (from tensorflow.python.ops.nn_ops) is deprecated and will be removed in a future version.
Instructions for updating:

Future major versions of TensorFlow will allow gradients to flow
into the labels input on backprop by default.

See `tf.nn.softmax_cross_entropy_with_logits_v2`.
```

```
cost = Tensor("Mean:0", shape=(), dtype=float32)
```

构建模型

In [25]:
```
def model(X_train,Y_train,X_test,Y_test,
          learning_rate=0.0001,num_epochs=1500,minibatch_size=32,
          print_cost=True,is_plot=True):
    """
    实现一个三层的TensorFlow神经网络：LINEAR->RELU->LINEAR->RELU->LINEAR->SOFTMAX

    参数：
        X_train - 训练集，维度为（输入大小（输入节点数量） = 12288, 样本数量 = 1080）
        Y_train - 训练集分类数量，维度为（输出大小(输出节点数量) = 6, 样本数量 = 1080）
        X_test - 测试集，维度为（输入大小（输入节点数量） = 12288, 样本数量 = 120）
        Y_test - 测试集分类数量，维度为（输出大小(输出节点数量) = 6, 样本数量 = 120）
        learning_rate - 学习速率
        num_epochs - 整个训练集的遍历次数
        mini_batch_size - 每个小批量数据集的大小
        print_cost - 是否打印成本，每100代打印一次
        is_plot - 是否绘制曲线图

    返回：
        parameters - 学习后的参数

    """
    ops.reset_default_graph()                    #能够重新运行模型而不覆盖tf变量
    tf.set_random_seed(1)
    seed = 3
    (n_x , m)  = X_train.shape                   #获取输入节点数量和样本数
    n_y = Y_train.shape[0]                       #获取输出节点数量
    costs = []                                   #成本集

    #给X和Y创建placeholder
    X,Y = create_placeholders(n_x,n_y)

    #初始化参数
    parameters = initialize_parameters()

    #前向传播
    Z3 = forward_propagation(X,parameters)

    #计算成本
    cost = compute_cost(Z3,Y)

    ### Start Code Here ###
    #实现反向传播，使用Adam优化
    optimizer = tf.compat.v1.train.AdamOptimizer(learning_rate=learning_rate).minimize(cost)

    ### End Code Here ###
```

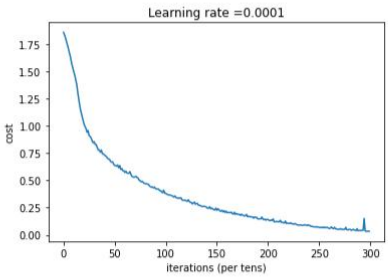
```
    #实现反向传播，使用Adam优化
    optimizer = tf.compat.v1.train.AdamOptimizer(learning_rate=learning_rate).minimize(cost)

    ### End Code Here ###

    #初始化所有的变量
    init = tf.global_variables_initializer()

    #开始会话并计算
    with tf.Session() as sess:
        #初始化
        sess.run(init)

        #正常训练的循环
        ### Start Code Here ###
        #设计并实现一个模型训练的过程，使用batch来控制每次训练的样本个数，并在一定的次数打印当前的cost值
        for epoch in range(num_epochs):

            epoch_cost = 0  #每代的成本
            num_minibatches = int(m / minibatch_size)    #minibatch的总数量
            seed = seed + 1
            minibatches = tf_utils.random_mini_batches(X_train,Y_train,minibatch_size,seed)

            for minibatch in minibatches:

                #选择一个minibatch
                (minibatch_X,minibatch_Y) = minibatch

                #数据已经准备好了，开始运行session
                _ , minibatch_cost = sess.run([optimizer,cost],feed_dict={X:minibatch_X,Y:minibatch_Y})

                #计算这个minibatch在这一代中所占的误差
                epoch_cost = epoch_cost + minibatch_cost / num_minibatches

            #记录并打印成本
            ## 记录成本
            if epoch % 5 == 0:
                costs.append(epoch_cost)
                #是否打印：
                if print_cost and epoch % 100 == 0:
                        print("epoch = " + str(epoch) + "    epoch_cost = " + str(epoch_cost))

        ### End Code Here ###


        #是否绘制图谱
        if is_plot:
            plt.plot(np.squeeze(costs))
```

```
    #是否绘制图谱
    if is_plot:
        plt.plot(np.squeeze(costs))
        plt.ylabel('cost')
        plt.xlabel('iterations (per tens)')
        plt.title("Learning rate =" + str(learning_rate))
        plt.show()

    #保存学习后的参数
    parameters = sess.run(parameters)
    print("参数已经保存到session。")

    #计算当前的预测结果
    correct_prediction = tf.equal(tf.argmax(Z3),tf.argmax(Y))

    #计算准确率
    accuracy = tf.reduce_mean(tf.cast(correct_prediction,"float"))

    print("训练集的准确率: ", accuracy.eval({X: X_train, Y: Y_train}))
    print("测试集的准确率:", accuracy.eval({X: X_test, Y: Y_test}))

    return parameters
```

In [26]:

```
#开始时间
start_time = time.clock()
#开始训练
parameters = model(X_train, Y_train, X_test, Y_test)
#结束时间
end_time = time.clock()
#计算时差
print("CPU的执行时间 = " + str(end_time - start_time) + " 秒" )
```

```
epoch = 0     epoch_cost = 1.8557019342075698
epoch = 100     epoch_cost = 1.0164577870657951
epoch = 200     epoch_cost = 0.7331022918224335
epoch = 300     epoch_cost = 0.5729406358617726
epoch = 400     epoch_cost = 0.468689679647937
epoch = 500     epoch_cost = 0.3810609809376977
epoch = 600     epoch_cost = 0.31380960887128656
epoch = 700     epoch_cost = 0.25419822877103626
epoch = 800     epoch_cost = 0.20379560215003564
epoch = 900     epoch_cost = 0.1663456656264537
epoch = 1000     epoch_cost = 0.14140377329154447
epoch = 1100     epoch_cost = 0.10749359993320526
epoch = 1200     epoch_cost = 0.08623977846494228
epoch = 1300     epoch_cost = 0.059411559423262435
epoch = 1400     epoch_cost = 0.05221025527200915
```

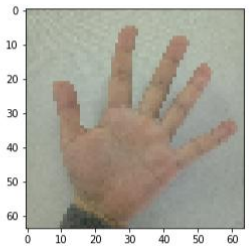



```
参数已经保存到session。
训练集的准确率:  0.9990741
测试集的准确率: 0.71666664
CPU的执行时间 = 2236.185655 秒
```

测试

In [27]:
```
import matplotlib.pyplot as plt # plt 用于显示图片
import matplotlib.image as mpimg # mpimg 用于读取图片
import numpy as np


my_image1 = "5.png"                                          #定义图片名称
image1 = mpimg.imread(my_image1)                             #读取图片
plt.imshow(image1)                                           #显示图片
my_image1 = image1.reshape(1,64 * 64 * 3).T                  #重构图片
my_image_prediction = tf_utils.predict(my_image1, parameters)  #开始预测
print("预测结果: y = " + str(np.squeeze(my_image_prediction)))
```

```
预测结果: y = 5
```



4. 深度神经网络的搭建与应用

In [1]:
```
import numpy as np
import h5py
import matplotlib.pyplot as plt
import testCases3
from dnn_utils import sigmoid, sigmoid_backward, relu, relu_backward
import lr_utils
```

【练习】初始化参数

对于一个两层的神经网络结构而言，模型结构是线性->ReLU->线性->sigmod函数。

In [3]:
```
def initialize_parameters(n_x,n_h,n_y):
    """
    此函数是为了初始化两层网络参数而使用的函数。
    参数:
        n_x - 输入层节点数量
        n_h - 隐藏层节点数量
        n_y - 输出层节点数量

    返回:
        parameters - 包含你的参数的python字典:
            W1 - 权重矩阵,维度为 (n_h, n_x)
            b1 - 偏向量, 维度为 (n_h, 1)
            W2 - 权重矩阵, 维度为 (n_y, n_h)
            b2 - 偏向量, 维度为 (n_y, 1)

    """
    W1 = np.random.randn(n_h, n_x) * 0.01
    b1 = np.zeros((n_h, 1))
    W2 = np.random.randn(n_y, n_h) * 0.01
    b2 = np.zeros((n_y, 1))



    #使用断言确保我的数据格式是正确的
    assert(W1.shape == (n_h, n_x))
    assert(b1.shape == (n_h, 1))
    assert(W2.shape == (n_y, n_h))
    assert(b2.shape == (n_y, 1))

    parameters = {"W1": W1,
                  "b1": b1,
                  "W2": W2,
                  "b2": b2}

    return parameters
```

In [4]:
```
print("==============测试initialize_parameters==============")
parameters = initialize_parameters(3,2,1)
print("W1 = " + str(parameters["W1"]))
print("b1 = " + str(parameters["b1"]))
print("W2 = " + str(parameters["W2"]))
print("b2 = " + str(parameters["b2"]))
```

```
==============测试initialize_parameters==============
W1 = [[ 0.01624345 -0.00611756 -0.00528172]
 [-0.01072969  0.00865408 -0.02301539]]
b1 = [[0.]
 [0.]]
W2 = [[ 0.01744812 -0.00761207]]
b2 = [[0.]]
```

In [5]:
```python
def initialize_parameters_deep(layers_dims):
    """
    此函数是为了初始化多层网络参数而使用的函数。
    参数：
        layers_dims - 包含我们网络中每个图层的节点数量的列表

    返回：
        parameters - 包含参数“W1”，“b1”，...，“WL”，“bL”的字典：
                     W1 - 权重矩阵，维度为（layers_dims [1]，layers_dims [1-1]）
                     b1 - 偏向量，维度为（layers_dims [1]，1）
    """
    np.random.seed(3)
    parameters = {}
    L = len(layers_dims)

    for l in range(1,L):
        ### Start Code Here ###
        #初始化参数
        parameters["W" + str(l)] = np.random.randn(layers_dims[l], layers_dims[l - 1]) / np.sqrt(layers_dims[l - 1])
        parameters["b" + str(l)] = np.zeros((layers_dims[l], 1))

        ### End Code Here ###


        #确保我要的数据的格式是正确的
        assert(parameters["W" + str(l)].shape == (layers_dims[l], layers_dims[l-1]))
        assert(parameters["b" + str(l)].shape == (layers_dims[l], 1))

    return parameters
```

In [6]:
```python
#测试initialize_parameters_deep
print("==============测试initialize_parameters_deep==============")
layers_dims = [5,4,3]
parameters = initialize_parameters_deep(layers_dims)
print("W1 = " + str(parameters["W1"]))
print("b1 = " + str(parameters["b1"]))
print("W2 = " + str(parameters["W2"]))
print("b2 = " + str(parameters["b2"]))
```

```
==============测试initialize_parameters_deep==============
W1 = [[ 0.79989897  0.19521314  0.04315498 -0.83337927 -0.12405178]
 [-0.15865304 -0.03700312 -0.28040323 -0.01959608 -0.21341839]
 [-0.58757818  0.39561516  0.39413741  0.76454432  0.02237573]
 [-0.18097724 -0.24389238 -0.69160568  0.43932807 -0.49241241]]
b1 = [[0.]
 [0.]
 [0.]
 [0.]]
W2 = [[-0.59252326 -0.10282495  0.74307418  0.11835813]
 [-0.51189257 -0.3564966   0.31262248 -0.08025668]
 [-0.38441818 -0.11501536  0.37252813  0.98805539]]
b2 = [[0.]
 [0.]
 [0.]]
```

【练习】线性部分LINEAR

In [7]:
```python
def linear_forward(A,W,b):
    """
    实现前向传播的线性部分。

    参数：
        A - 来自上一层（或输入数据）的激活，维度为(上一层的节点数量，示例的数量)
        W - 权重矩阵，numpy数组，维度为（当前图层的节点数量，前一图层的节点数量）
        b - 偏向量，numpy向量，维度为（当前图层节点数量，1）

    返回：
         Z - 激活功能的输入，也称为预激活参数
         cache - 一个包含“A”，“W”和“b”的字典，存储这些变量以有效地计算后向传递
    """
    ### Start Code Here ###
    #实现A的线性计算，Z=W*A+b
    Z = np.dot(W,A) + b

    ### End Code Here ###

    assert(Z.shape == (W.shape[0],A.shape[1]))
    cache = (A,W,b)

    return Z,cache
```

In [9]:
```python
#测试linear_forward
print("==============测试linear_forward==============")
A,W,b = testCases3.linear_forward_test_case()
Z,linear_cache = linear_forward(A,W,b)
print("Z = " + str(Z))
```

```
==============测试linear_forward==============
Z = [[ 3.26295337 -1.23429987]]
```

In [10]:
```python
def linear_activation_forward(A_prev,W,b,activation):
    """
    实现LINEAR-> ACTIVATION 这一层的前向传播

    参数:
        A_prev - 来自上一层（或输入层）的激活，维度为(上一层的节点数量，示例数）
        W - 权重矩阵，numpy数组，维度为（当前层的节点数量，前一层的大小）
        b - 偏向量，numpy阵列，维度为（当前层的节点数量，1）
        activation - 选择在此层中使用的激活函数名，字符串类型，【"sigmoid" | "relu"】

    返回:
        A - 激活函数的输出，也称为激活后的值
        cache - 一个包含“linear_cache”和“activation_cache”的字典，我们需要存储它以有效地计算后向传递
    """

    if activation == "sigmoid":
        Z, linear_cache = linear_forward(A_prev, W, b)
        A, activation_cache = sigmoid(Z)
    elif activation == "relu":
        Z, linear_cache = linear_forward(A_prev, W, b)
        A, activation_cache = relu(Z)

    assert(A.shape == (W.shape[0],A_prev.shape[1]))
    cache = (linear_cache,activation_cache)

    return A,cache
```

In [11]:
```python
#测试linear_activation_forward
print("==========测试linear_activation_forward==========")
A_prev, W,b = testCases3.linear_activation_forward_test_case()

A, linear_activation_cache = linear_activation_forward(A_prev, W, b, activation = "sigmoid")
print("sigmoid, A = " + str(A))

A, linear_activation_cache = linear_activation_forward(A_prev, W, b, activation = "relu")
print("ReLU, A = " + str(A))
```

```
==========测试linear_activation_forward==========
sigmoid, A = [[0.96890023 0.11013289]]
ReLU, A = [[3.43896131 0.        ]]
```

In [12]:
```python
def L_model_forward(X,parameters):
    """
    实现[LINEAR-> RELU] *（L-1） - > LINEAR-> SIGMOID计算前向传播，也就是多层网络的前向传播，为后面每一层都执行LINEAR和ACTIVATION

    参数:
        X - 数据，numpy数组，维度为（输入节点数量，示例数）
        parameters - initialize_parameters_deep（）的输出

    返回:
        AL - 最后的激活值
        caches - 包含以下内容的缓存列表:
                linear_relu_forward（）的每个cache（有L-1个，索引为从0到L-2）
                linear_sigmoid_forward（）的cache（只有一个，索引为L-1）
    """
    caches = []
    A = X
    L = len(parameters) // 2

    ### Start Code Here ###
    #设计并实现多层网络模型的前向传播
    for l in range(1,L):
        A_prev = A
        A, cache = linear_activation_forward(A_prev,parameters['W'+str(l)],parameters['b' + str(l)], "relu")
        caches.append(cache)

    AL, cache = linear_activation_forward(A, parameters['W' + str(L)], parameters['b' + str(L)], "sigmoid")
    caches.append(cache)
    ### End Code Here ###


    assert(AL.shape == (1,X.shape[1]))

    return AL,caches
```

In [13]:
```python
#测试L_model_forward
print("==========测试L_model_forward==========")
X,parameters = testCases3.L_model_forward_test_case()
AL,caches = L_model_forward(X,parameters)
print("AL = " + str(AL))
print("caches 的长度为 = " + str(len(caches)))
```

```
==========测试L_model_forward==========
AL = [[0.17007265 0.2524272 ]]
caches 的长度为 = 2
```

In [16]:
```python
def linear_backward(dZ,cache):
    """
    为单层实现反向传播的线性部分（第L层）

    参数:
         dZ - 相对于（当前第l层的）线性输出的成本梯度
         cache - 来自当前层前向传播的值的元组（A_prev，W，b）

    返回:
         dA_prev - 相对于激活（前一层l-1）的成本梯度，与A_prev维度相同
         dW - 相对于W（当前层l）的成本梯度，与W的维度相同
         db - 相对于b（当前层l）的成本梯度，与b维度相同
    """
    A_prev, W, b = cache
    m = A_prev.shape[1]
    dW = np.dot(dZ, A_prev.T) / m
    db = np.sum(dZ, axis=1, keepdims=True) / m
    dA_prev = np.dot(W.T, dZ)

    assert (dA_prev.shape == A_prev.shape)
    assert (dW.shape == W.shape)
    assert (db.shape == b.shape)

    return dA_prev, dW, db
```

In [17]:
```python
#测试linear_backward
print("==========测试linear_backward==========")
dZ, linear_cache = testCases3.linear_backward_test_case()

dA_prev, dW, db = linear_backward(dZ, linear_cache)
print ("dA_prev = "+ str(dA_prev))
print ("dW = " + str(dW))
print ("db = " + str(db))
```

```
==========测试linear_backward==========
dA_prev = [[ 0.51822968 -0.19517421]
 [-0.40506361  0.15255393]
 [ 2.37496825 -0.89445391]]
dW = [[-0.10076895  1.40685096  1.64992505]]
db = [[0.50629448]]
```

In [18]:
```python
def linear_activation_backward(dA,cache,activation="relu"):
    """
    实现LINEAR-> ACTIVATION层的后向传播。

    参数:
         dA - 当前层l的激活后的梯度值
         cache - 我们存储的用于有效计算反向传播的值的元组（值为linear_cache，activation_cache）
         activation - 要在此层中使用的激活函数名，字符串类型，【"sigmoid" | "relu"】
    返回:
         dA_prev - 相对于激活（前一层l-1）的成本梯度值，与A_prev维度相同
         dW - 相对于W（当前层l）的成本梯度值，与W的维度相同
         db - 相对于b（当前层l）的成本梯度值，与b的维度相同
    """
    linear_cache, activation_cache = cache
    if activation == "relu":
        dZ = relu_backward(dA, activation_cache)
        dA_prev, dW, db = linear_backward(dZ, linear_cache)
    elif activation == "sigmoid":
        dZ = sigmoid_backward(dA, activation_cache)
        dA_prev, dW, db = linear_backward(dZ, linear_cache)

    return dA_prev,dW,db
```

In [19]:
```python
# 测试linear_activation_backward
print("==========测试linear_activation_backward==========")
AL, linear_activation_cache = testCases3.linear_activation_backward_test_case()

dA_prev, dW, db = linear_activation_backward(
    AL, linear_activation_cache, activation="sigmoid")
print("sigmoid:")
print("dA_prev = " + str(dA_prev))
print("dW = " + str(dW))
print("db = " + str(db) + "\n")
```

In [19]:
```python
# 测试linear_activation_backward
print("==============测试linear_activation_backward==============")
AL, linear_activation_cache = testCases3.linear_activation_backward_test_case()

dA_prev, dW, db = linear_activation_backward(
    AL, linear_activation_cache, activation="sigmoid")
print("sigmoid:")
print("dA_prev = " + str(dA_prev))
print("dW = " + str(dW))
print("db = " + str(db) + "\n")

dA_prev, dW, db = linear_activation_backward(
    AL, linear_activation_cache, activation="relu")
print("relu:")
print("dA_prev = " + str(dA_prev))
print("dW = " + str(dW))
print("db = " + str(db))
```

```
==============测试linear_activation_backward==============
sigmoid:
dA_prev = [[ 0.11017994  0.01105339]
 [ 0.09466817  0.00949723]
 [-0.05743092 -0.00576154]]
dW = [[ 0.10266786  0.09778551 -0.01968084]]
db = [[-0.05729622]]

relu:
dA_prev = [[ 0.44090989  0.        ]
 [ 0.37883606  0.        ]
 [-0.2298228   0.        ]]
dW = [[ 0.44513824  0.37371418 -0.10478989]]
db = [[-0.20837892]]
```

In [20]:
```python
dAL = - (np.divide(Y.T, AL) - np.divide(1 - Y.T, 1 - AL))
```

【练习】实现多层网络模型的反向传播

In [22]:
```python
def L_model_backward(AL, Y, caches):
    """
    对[LINEAR-> RELU] *（L-1） - > LINEAR - > SIGMOID组执行反向传播，就是多层网络的向后传播

    参数：
     AL - 概率向量，正向传播的输出（L_model_forward（））
     Y - 标签向量（例如：如果不是猫，则为0，如果是猫则为1），维度为（1，数量）
     caches - 包含以下内容的cache列表：
                 linear_activation_forward（"relu"）的cache，不包含输出层
                 linear_activation_forward（"sigmoid"）的cache

    返回：
     grads - 具有梯度值的字典
             grads [“dA”+ str（l）] = ...
             grads [“dW”+ str（l）] = ...
             grads [“db”+ str（l）] = ...
    """
    grads = {}
    L = len(caches)
    m = AL.shape[1]
    Y = Y.reshape(AL.shape)
    dAL = - (np.divide(Y, AL) - np.divide(1 - Y, 1 - AL))

    current_cache = caches[L-1]
    grads["dA" + str(L)], grads["dW" + str(L)], grads["db" + str(L)
                                                    ] = linear_activation_backward(dAL, current_cache, "sigmoid")

    ### Start Code Here ###
    #设计并实现计算1至L-1层的反向传播结果并保存至grads
    for l in reversed(range(L-1)):
        current_cache = caches[l]
        dA_prev_temp, dW_temp, db_temp = linear_activation_backward(grads["dA" + str(l + 2)], current_cache, "relu")
        grads["dA" + str(l + 1)] = dA_prev_temp
        grads["dW" + str(l + 1)] = dW_temp
        grads["db" + str(l + 1)] = db_temp

    ### End Code Here ###


    return grads
```

In [23]:
```python
#测试L_model_backward
print("==============测试L_model_backward==============")
AL, Y_assess, caches = testCases3.L_model_backward_test_case()
grads = L_model_backward(AL, Y_assess, caches)
print ("dW1 = "+ str(grads["dW1"]))
print ("db1 = "+ str(grads["db1"]))
print ("dA1 = "+ str(grads["dA1"]))
```

```
==============测试L_model_backward==============
dW1 = [[0.41010002 0.07807203 0.13798444 0.10502167]
 [0.         0.         0.         0.        ]
 [0.05283652 0.01005865 0.01777766 0.0135308 ]]
db1 = [[-0.22007063]
 [ 0.        ]
 [-0.02835349]]
dA1 = [[ 0.          0.52257901]
 [ 0.         -0.3269206 ]
 [ 0.         -0.32070404]
 [ 0.         -0.74079187]]
```

更新参数

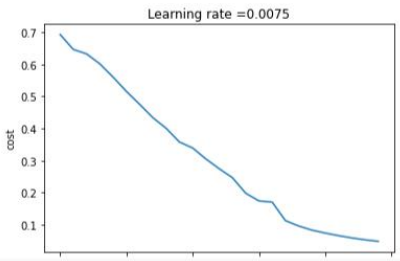
In [24]:
```
def update_parameters(parameters, grads, learning_rate):
    """
    使用梯度下降更新参数

    参数:
     parameters - 包含你的参数的字典
     grads - 包含梯度值的字典，是L_model_backward的输出

    返回:
     parameters - 包含更新参数的字典
                  参数[“W” + str（l）] = ...
                  参数[“b” + str（l）] = ...
    """
    L = len(parameters) // 2 #整除
    for l in range(L):
        parameters["W" + str(l + 1)] = parameters["W" + str(l + 1)] - learning_rate * grads["dW" + str(l + 1)]
        parameters["b" + str(l + 1)] = parameters["b" + str(l + 1)] - learning_rate * grads["db" + str(l + 1)]

    return parameters
```

In [25]:
```
#测试update_parameters
print("==============测试update_parameters==============")
parameters, grads = testCases3.update_parameters_test_case()
parameters = update_parameters(parameters, grads, 0.1)

print ("W1 = "+ str(parameters["W1"]))
print ("b1 = "+ str(parameters["b1"]))
print ("W2 = "+ str(parameters["W2"]))
print ("b2 = "+ str(parameters["b2"]))
```

```
==============测试update_parameters==============
W1 = [[-0.59562069 -0.09991781 -2.14584584  1.82662008]
 [-1.76569676 -0.80627147  0.51115557 -1.18258802]
 [-1.0535704  -0.86128581  0.68284052  2.20374577]]
b1 = [[-0.04659241]
 [-1.28888275]
 [ 0.53405496]]
W2 = [[-0.55569196  0.0354055   1.32964895]]
b2 = [[-0.84610769]]
```

```
n_h = 7
n_y = 1
layers_dims = (n_x,n_h,n_y)

parameters = two_layer_model(train_x, train_set_y, layers_dims = (n_x, n_h, n_y), num_iterations = 2500, print_cost=True,isPlot=True)
```

```
第 0 次迭代，成本值为： 0.6930497356599891
第 100 次迭代，成本值为： 0.6464320953428849
第 200 次迭代，成本值为： 0.6325140647912678
第 300 次迭代，成本值为： 0.6015024920354665
第 400 次迭代，成本值为： 0.5601966311605748
第 500 次迭代，成本值为： 0.515830477276473
第 600 次迭代，成本值为： 0.4754901313943326
第 700 次迭代，成本值为： 0.4339163151225749
第 800 次迭代，成本值为： 0.4007977536203885
第 900 次迭代，成本值为： 0.3580705011323798
第 1000 次迭代，成本值为： 0.3394281538366414
第 1100 次迭代，成本值为： 0.30527536361962654
第 1200 次迭代，成本值为： 0.2749137728213014
第 1300 次迭代，成本值为： 0.24681768210614857
第 1400 次迭代，成本值为： 0.19850735037466108
第 1500 次迭代，成本值为： 0.17448318112556654
第 1600 次迭代，成本值为： 0.17080762978096026
第 1700 次迭代，成本值为： 0.11306524562164742
第 1800 次迭代，成本值为： 0.09629426845937165
第 1900 次迭代，成本值为： 0.0834261795972687
第 2000 次迭代，成本值为： 0.07439078704319083
第 2100 次迭代，成本值为： 0.06630748132267934
第 2200 次迭代，成本值为： 0.05919329501038171
第 2300 次迭代，成本值为： 0.05336140348560557
第 2400 次迭代，成本值为： 0.0485547856287702
```



In [30]:
```python
def predict(X, y, parameters):
    """
    该函数用于预测L层神经网络的结果，当然也包含两层

    参数:
     X - 测试集
     y - 标签
     parameters - 训练模型的参数

    返回:
     p - 给定数据集X的预测
    """

    m = X.shape[1]
    n = len(parameters) // 2 # 神经网络的层数
    p = np.zeros((1,m))

    #根据参数前向传播
    probas, caches = L_model_forward(X, parameters)

    for i in range(0, probas.shape[1]):
        if probas[0,i] > 0.5:
            p[0,i] = 1
        else:
            p[0,i] = 0

    print("准确度为: " + str(float(np.sum((p == y))/m)))

    return p
```

预测函数构建好了我们就开始预测，查看训练集和测试集的准确性：

In [31]:
```python
predictions_train = predict(train_x, train_y, parameters) #训练集
predictions_test = predict(test_x, test_y, parameters) #测试集
```

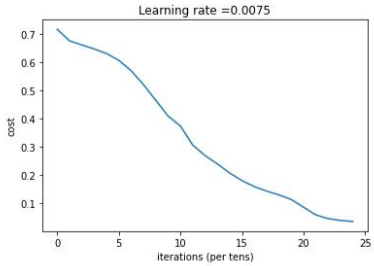
```
准确度为: 1.0
准确度为: 0.72
```

【练习】搭建多层神经网络

In [32]:
```python
def L_layer_model(X, Y, layers_dims, learning_rate=0.0075, num_iterations=3000, print_cost=False,isPlot=True):
    """
    实现一个L层神经网络：[LINEAR-> RELU] *（L-1） - > LINEAR-> SIGMOID。

    参数:
        X - 输入的数据，维度为(n_x，例子数)
        Y - 标签，向量，0为非猫，1为猫，维度为(1,数量)
        layers_dims - 层数的向量，维度为(n_y,n_h,···,n_h,n_y)
        learning_rate - 学习率
        num_iterations - 迭代的次数
        print_cost - 是否打印成本值，每100次打印一次
        isPlot - 是否绘制出误差值的图谱

    返回:
     parameters - 模型学习的参数。 然后他们可以用来预测。
    """
    ### Start Code Here ###
    #设计并实现一个多层神经网络
    np.random.seed(1)
    costs = []

    parameters = initialize_parameters_deep(layers_dims)

    for i in range(0,num_iterations):
        AL , caches = L_model_forward(X,parameters)

        cost = compute_cost(AL,Y)

        grads = L_model_backward(AL,Y,caches)

        parameters = update_parameters(parameters,grads,learning_rate)

        if i % 100 == 0:
            costs.append(cost)
            if print_cost:
                print("第", i ,"次迭代，成本值为：" ,np.squeeze(cost))

    ### End Code Here ###

    #迭代完成，根据条件绘制图
    if isPlot:
        plt.plot(np.squeeze(costs))
        plt.ylabel('cost')
```

```
In [34]: layers_dims = [12288, 20, 7, 5, 1] #  5-layer model
         parameters = L_layer_model(train_x, train_y, layers_dims, num_iterations = 2500, print_cost = True,isPlot=True)
```

```
第 0 次迭代，成本值为： 0.715731513413713
第 100 次迭代，成本值为： 0.6747377593469114
第 200 次迭代，成本值为： 0.6603365433622127
第 300 次迭代，成本值为： 0.6462887802148751
第 400 次迭代，成本值为： 0.6298131216927773
第 500 次迭代，成本值为： 0.606005622926534
第 600 次迭代，成本值为： 0.5690041263975134
第 700 次迭代，成本值为： 0.5197965350438059
第 800 次迭代，成本值为： 0.46415716786282285
第 900 次迭代，成本值为： 0.40842030048298916
第 1000 次迭代，成本值为： 0.3731549921606903
第 1100 次迭代，成本值为： 0.3057237457304711
第 1200 次迭代，成本值为： 0.26810152847740837
第 1300 次迭代，成本值为： 0.23872474827672635
第 1400 次迭代，成本值为： 0.20632263257914712
第 1500 次迭代，成本值为： 0.17943886927493577
第 1600 次迭代，成本值为： 0.15798735818801465
第 1700 次迭代，成本值为： 0.14240413012274267
第 1800 次迭代，成本值为： 0.12865165997887576
第 1900 次迭代，成本值为： 0.11244314998160974
第 2000 次迭代，成本值为： 0.08505631034976424
第 2100 次迭代，成本值为： 0.05758391198612545
第 2200 次迭代，成本值为： 0.04456753454697131
第 2300 次迭代，成本值为： 0.038082751665992724
第 2400 次迭代，成本值为： 0.03441074901841349
```




```
In [35]: pred_train = predict(train_x, train_y, parameters) #训练集
         pred_test = predict(test_x, test_y, parameters) #测试集
```

```
准确度为: 0.9952153110047847
准确度为: 0.78
```

```
In [36]: def print_mislabeled_images(classes, X, y, p):
             """
             绘制预测和实际不同的图像。
                 X - 数据集
                 y - 实际的标签
                 p - 预测
             """
             a = p + y
             mislabeled_indices = np.asarray(np.where(a == 1))
             plt.rcParams['figure.figsize'] = (40.0, 40.0) # set default size of plots
             num_images = len(mislabeled_indices[0])
             for i in range(num_images):
                 index = mislabeled_indices[1][i]

                 plt.subplot(2, num_images, i + 1)
                 plt.imshow(X[:,index].reshape(64,64,3), interpolation='nearest')
                 plt.axis('off')
                 plt.title("Prediction: " + classes[int(p[0,index])].decode("utf-8") + " \n Class: " + classes[y[0,index]].decode("utf-8"))


         print_mislabeled_images(classes, test_x, test_y, pred_test)
```

五、实验总结

通过本次实验，我了解了 tensorflow 及其变量，并学习了如何应用 tensorflow，学习了如何进行深度神经网络的搭建与应用和卷积神经网络的搭建与训练，在进行实验时遇到了缺少数据集的问题，通过参考网上资料后，下载了数据集并上传至平台，解决了这个问题